I. Dokumentacja do lab. nr 2 - "Paradygmaty obiektowości"

II. Roch Majkowski - grupa 1, semestr III

III. Przedmiot - "Programowanie obiektowe"

IV. Opis zadania do realizacji

Do zrealizowania były następujące zadania:

- Zaprojektuj klasę Book z polami title, author, available oraz metodami borrow() i returnBook().,
- Zaimplementuj klasę BankAccount z prywatnym polem balance i metodami deposit(double) oraz withdraw(double) z walidacją danych.,
- Utwórz hierarchię Vehicle -> Car i pokaż w main, które pola/metody są dziedziczone.,
- Dodaj klasę ElectricCar, która przesłania metodę getDescription() i zademonstruj polimorfizm na tablicy Vehicle[].

V. Technologie wykorzystane w zadaniu

- Java

VI. Realizacja zadania

1. Zadanie - klasa Book

Kod wykorzystany do rozwiązania zadania (zadań):

```
public class Book_Lab {

    public static class Book {
        private String title;
        private String author;
        private Boolean available = true;

        public Book(String title, String author) {
            this.title = title;
            this.author = author;
        }

        public String getTitle() {
            return title;
        }
        public String getAuthor() {
```

```
        return author;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public void Borrow(){
        if(available!=false) {
            available = false;
            System.out.println("Wypożyczyłeś książkę " + title + ", autorstwa " + author);
        } else {
            System.out.println("Książka jest już wypożyczona");
        }
    }

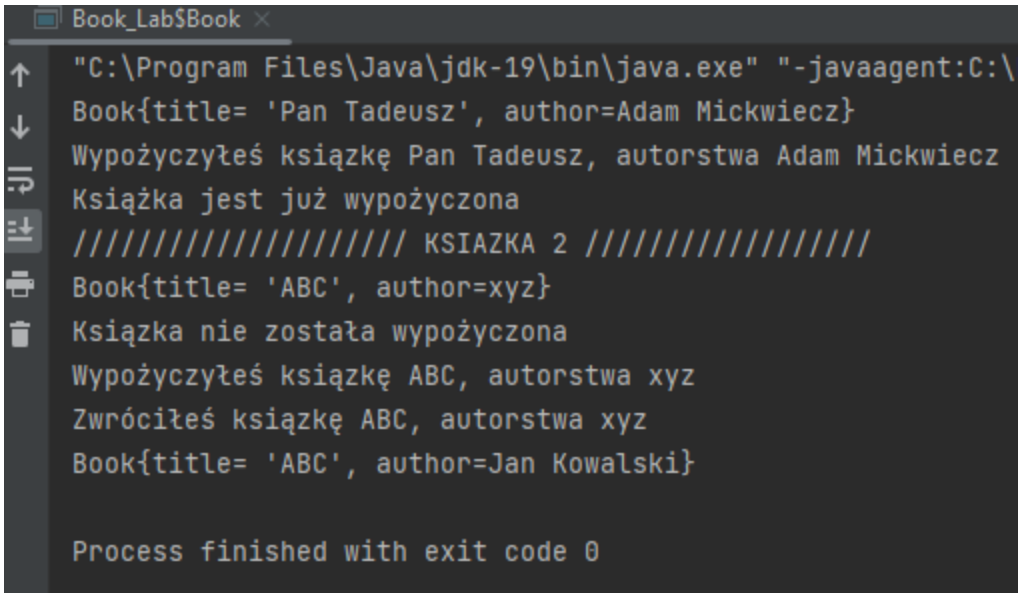
    public void ReturnBook(){
        if (available!=true) {
            available = true;
            System.out.println("Zwróciłeś książkę " + title + ", autorstwa " + author);
        } else {
            System.out.println("Książka nie została wypożyczona");
        }
    }

    @Override
    public String toString(){
        return "Book{title= '" + title + "', author=" + author + "}";
    }
    public static void main(String[] args) {
        Book book = new Book("Pan Tadeusz","Adam Mickwiecz");
        Book book1 = new Book("ABC", "xyz");
        System.out.println(book);
        book.Borrow();
        book.Borrow();

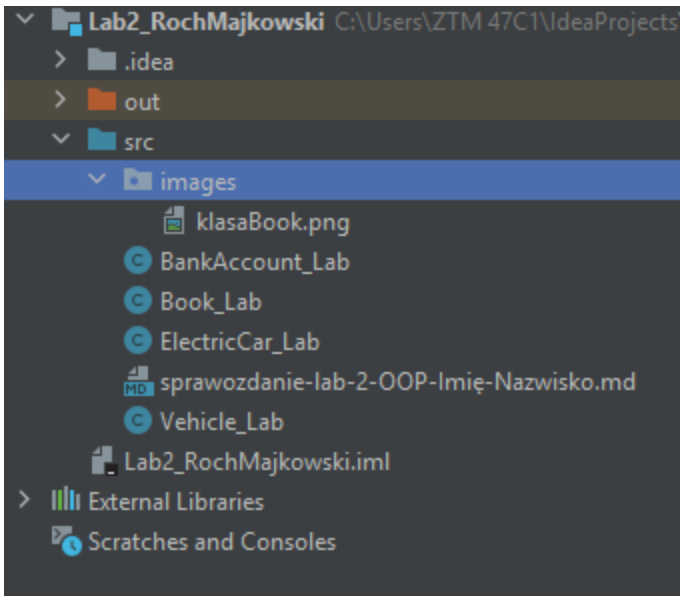
        System.out.println("////////////////////// KSIAZKA 2 //////////////////");
        System.out.println(book1);
        book1.ReturnBook();
        book1.Borrow();
        book1.ReturnBook();
        book1.setAuthor("Jan Kowalski");
        System.out.println(book1);
    }

}
}
```

1a. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



1b. Struktura projektu/programu:



2. Zadanie - klasa BankAccount

Kod wykorzystany do rozwiązania zadania (zadań):

```
public class BankAccount_Lab {
    public static class BankAccount {
        private double balance;

        public void deposit(double kwota) {
            if (kwota > 0) {
                this.balance += kwota;
                System.out.println("Deposited: "+ kwota);
            } else {
                System.out.println("Balance must be higher than 0!");
            }
        }
```

```
        public void withdraw(double kwota) {
            if (kwota > 0 && kwota <= balance) {
                this.balance -= kwota;
                System.out.println("Withdrawn: " + kwota);
            } else {
                System.out.println("No funds available or wrong number inputted");
            }
        }

        public double getBalance() {
            return balance;
        }

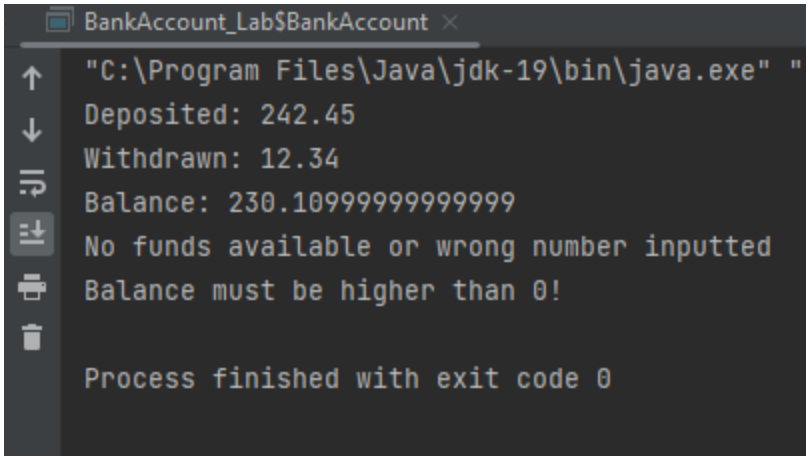
        public static void main(String[] args) {
            BankAccount account = new BankAccount();
            account.deposit(242.45);
            account.withdraw(12.34);
            System.out.println("Balance: "+ account.getBalance());
            account.withdraw(999.99);
            account.deposit(-199.99);

        }

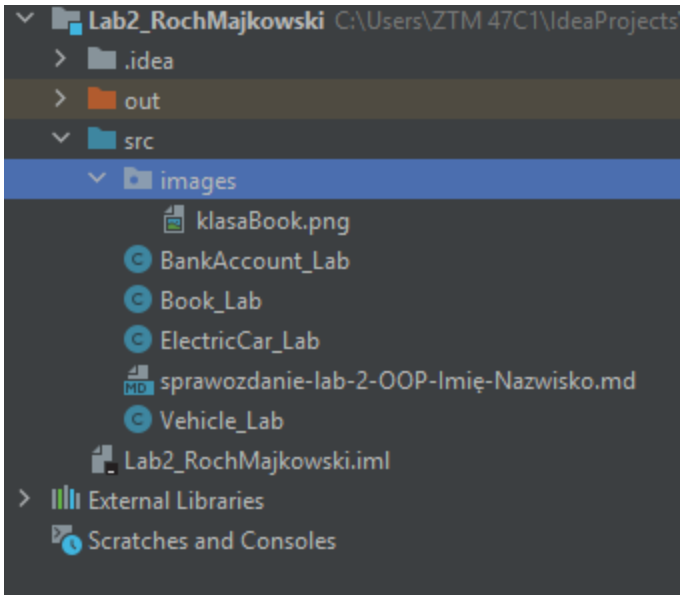
    }

}
```

2a. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



2b. Struktura projektu/programu:



3. Zadanie - Hierarchia Vehicle -> Car

Kod wykorzystany do rozwiązania zadania (zadań):

```
public class Vehicle_Lab {
    public static class Vehicle {
        String marka;
        int rok_produkcji;

        Vehicle(String marka, int rok_produkcji){
            this.marka = marka;
            this.rok_produkcji = rok_produkcji;
        }

        void getDescription() {
            System.out.println(
                    "Marka: "+ marka +"\n" +
                            "Rok produkcji: "+ rok_produkcji +"\n");
        }

        static class Car extends Vehicle {
            int liczba_drzwi;
            Car(String marka, int rok_produkcji, int liczba_drzwi) {
                super(marka, rok_produkcji);
                this.liczba_drzwi = liczba_drzwi;
            }

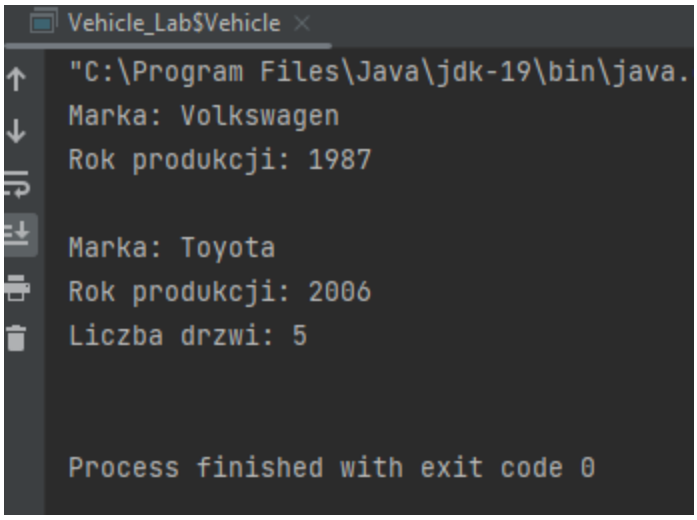
            @Override
            void getDescription() {
                System.out.println(
                        "Marka: "+ marka +"\n" +
                                "Rok produkcji: "+ rok_produkcji +"\n" +
                                "Liczba drzwi: "+ liczba_drzwi + "\n"); // Element jest ekskluzywny dla klasy C
            }

        }
```

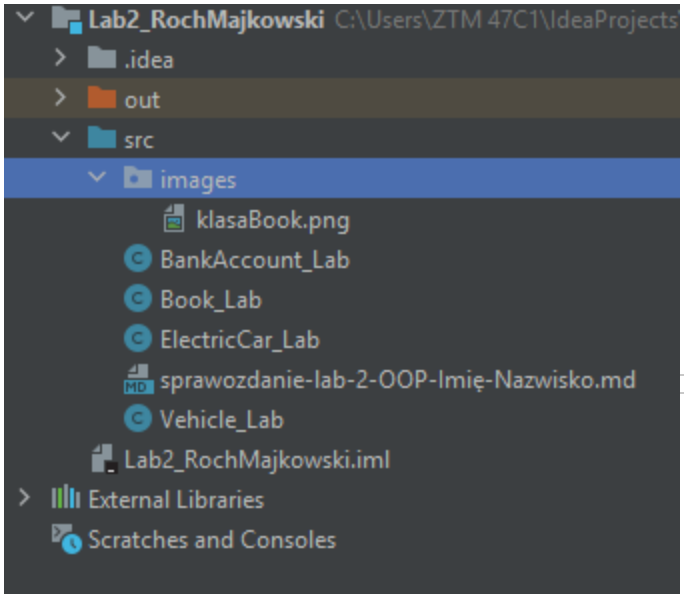
```
        public static void main(String[] args) {
            Vehicle vehicle = new Vehicle("Volkswagen",1987);
            vehicle.getDescription();
            Car car = new Car("Toyota", 2006, 5);
            car.getDescription();
        }

    }
}
```

3a. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



3b. Struktura projektu/programu:



4. Zadanie - ElectricCar

Kod wykorzystany do rozwiązania zadania (zadań):

```java
public class ElectricCar_Lab {
    public static class Vehicle {
        String marka;
        int rok_produkcji;

        Vehicle(String marka, int rok_produkcji){
            this.marka = marka;
            this.rok_produkcji = rok_produkcji;
        }

        void getDescription() {
            System.out.println(
                    "Marka: "+ marka +"\n" +
                            "Rok produkcji: "+ rok_produkcji +"\n");
        }

        static class ElectricCar extends Vehicle {
            int pojemnosc_baterii;
            ElectricCar(String marka, int rok_produkcji, int pojemnosc_baterii) {
                super(marka, rok_produkcji);
                this.pojemnosc_baterii = pojemnosc_baterii;
            }

            @Override
            void getDescription() {
                System.out.println(
                        "Marka: "+ marka +"\n" +
                                "Rok produkcji: "+ rok_produkcji +"\n" +
                                "Liczba drzwi: "+ pojemnosc_baterii + "\n"); // Element jest ekskluzywny dla kl
            }

        }

        static class Car extends Vehicle {
            int liczba_drzwi;
            Car(String marka, int rok_produkcji, int liczba_drzwi) {
                super(marka, rok_produkcji);
                this.liczba_drzwi = liczba_drzwi;
            }

            @Override
            void getDescription() {
                System.out.println(
                        "Marka: "+ marka +"\n" +
                                "Rok produkcji: "+ rok_produkcji +"\n" +
                                "Liczba drzwi: "+ liczba_drzwi + "\n"); // Element jest ekskluzywny dla klasy C
            }

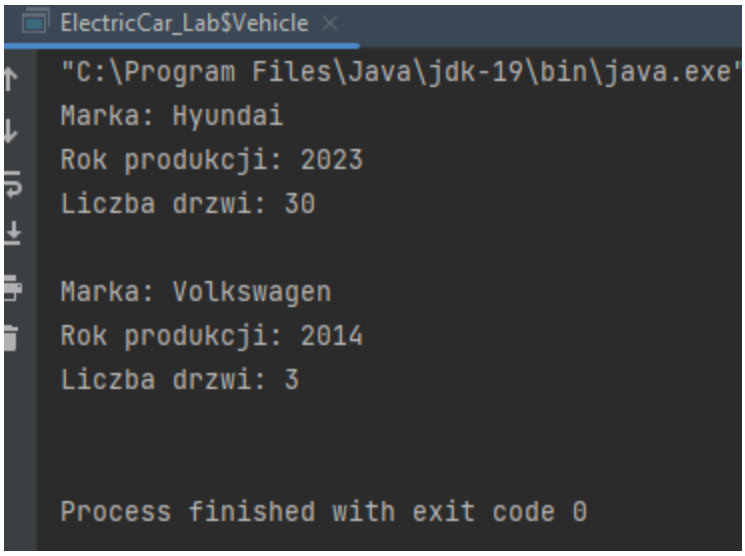
        }

        public static void main(String[] args) {
            Vehicle[] vehicles = {
                    new ElectricCar("Hyundai", 2023, 30),
                    new Car("Volkswagen", 2014, 3)
            };

            for(Vehicle vehicle : vehicles) {
                vehicle.getDescription();
            }
        }
```

```
    }
}
```

4a. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



4b. Struktura projektu/programu:

